Scalable Parallel-in-Time Integration for Equations of Motion

Nathaniel W. Chapman

Department of Computer Science
Central Washington University

2025-08-11

Table of Contents

Content

It's About Time

The main issue

Calculations take time...and we only have so much.

Minimizing Time

- Total runtime = $\frac{\text{time}}{\text{calculation}} * \text{number of calculations}$
- Wan to minimize runtime
- Minimizing time / calculation $\iff$ Speeding up sequential evaluation
- Maximizing calculations / time $\iff$ Adding more parallelization

Figure 1:

Parallelizing Time

- Physics depends on time and space
- Space has been parallelized
- Time has remained sequential
- Parallel-in-Time integration offers a new avenue to speedup

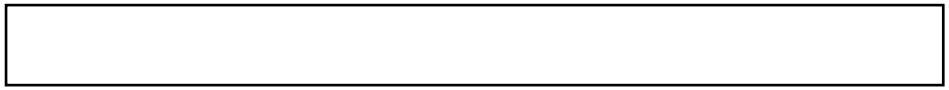


Figure 2:

Maximizing Resource Utilization

- Space discretized as much as it can e.g. constrained by CFL
- Spatial parallelization saturated
- Remaining cores sit idle

Figure 3:

More Threads is Always the Answer

- PinT scales with threads
- CPUs have ~10 threads
- GPUs have ~10,000 threads
- Distributed has ∞ threads

Figure 4:

This work

- Implement the Parareal Algorithm
- Use GPUs and distributed systems
- Simulate motion

Content

Growth of Parallel-in-Time Integration

- PinT is growing
- Not a new idea
- More cores ⟹ more interest
- On track for more growth

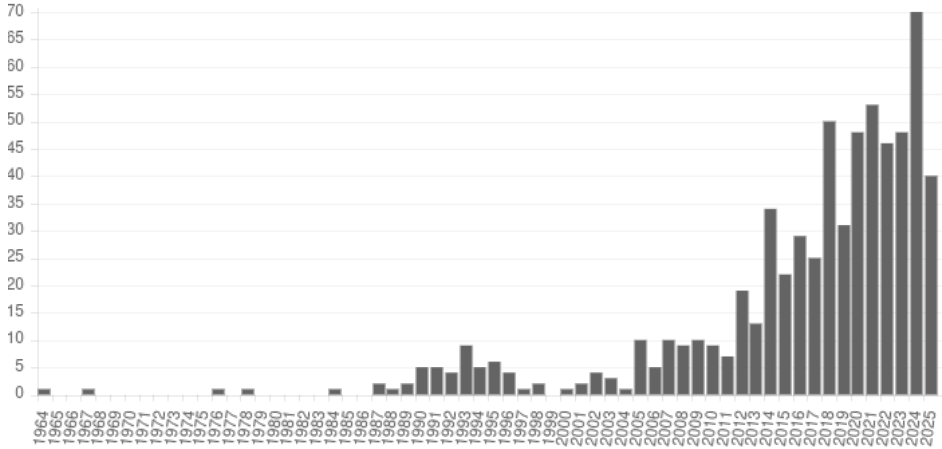


Figure 5: The number of papers published using parallel-in-time integration has been growing and even accelerating over the years.

Applications of Parallel-in-Time Integration

- Applications in many sciences
- **Physics**: E&M, Plasmas, Fluids, Materials, Astrophysics
- **Engineering**: Manufacturing
- **Operations**: Optimal control
- **Economics**: Game theory, Market dynamics
- **Biology**: Animal patterns, Blood flow in fish
- **Machine Learning**: Training neural networks

Methods of Parallel-in-Time Integration

- Specialized methods
- **General**: Parareal, PITA, PFASST, RIDC
- **Hyperbolic**: ParaDiag, MGRIT
- **Parabolic**: STMG, WRMG

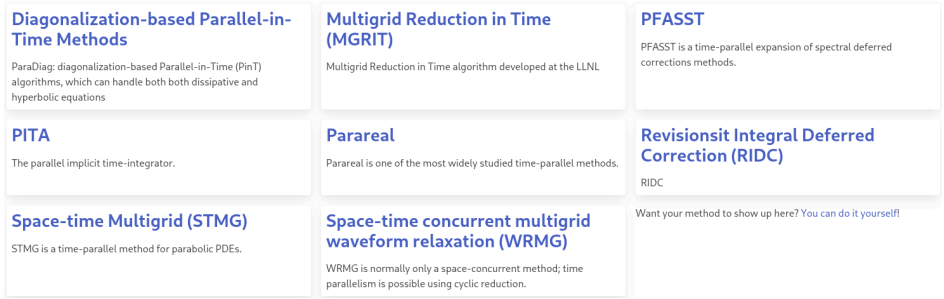


Figure 6: Several parallel-in-time methods have been developed. Each method aims to address limitations of others as well as providing new approaches in general.

Implementations of Parallel-in-Time Integration

- Different languages with different means of parallelism
- **Languages**: Fortran, C, C++, Python
- **Parallelism**: OpenMP, MPI
- **HPC**: Very few

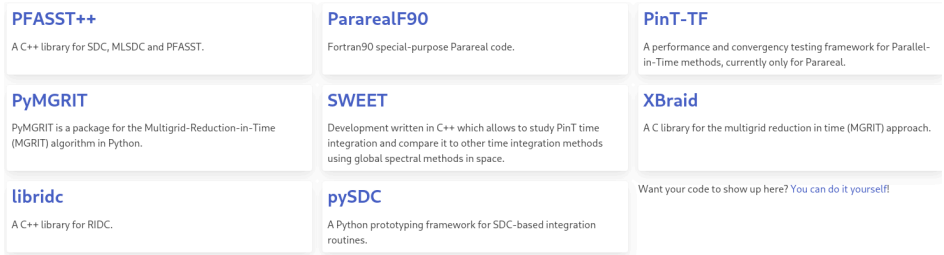


Figure 7: Several parallel-in-time methods have been implemented. Implementations have been developed in a variety of languages with a variety of approaches to parallelism.

Language of Parallel-in-Time Integration

- Traditional HPC languages have been used e.g. C/C++, Fortran
- Python has been used
- Julia has the speed of C with the ease of Python
- Julia natively supports parallel, GPU, and distributed
- PinT should be implemented in Julia

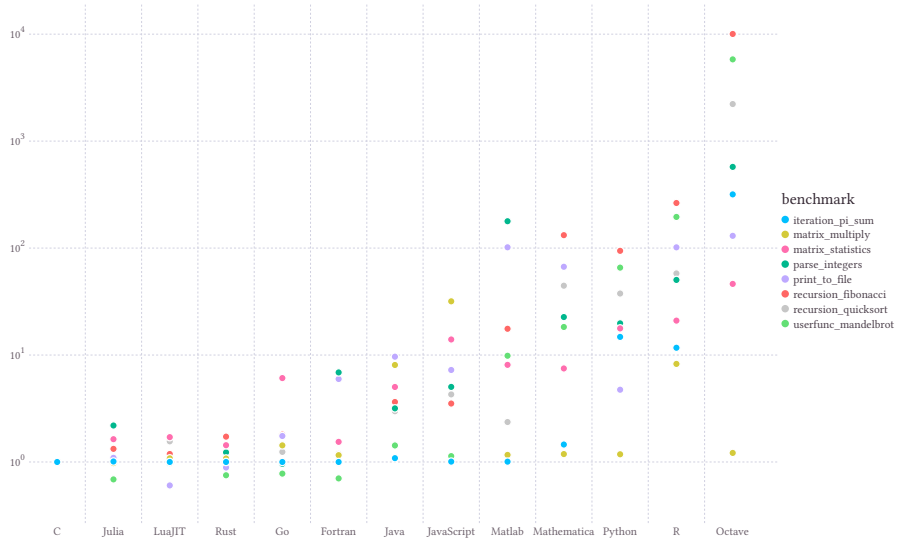


Figure 8: Several parallel-in-time methods have been implemented. Implementations have been developed in a variety of languages with a variety of approaches to parallelism.

This Contribution to Parallel-in-Time Integration

- **Application**: Physics of motion
- **Method**: Parareal
- **Parallelism**: Distributed GPUs
- **Language**: Julia

Content

Equations of Motion

- What is an equation of motion?
- **Physics**: A constraint on the dynamics of an object interacting with its environment
- **Math**: A second-order, hyperbolic, differential equation
- Solution is completely determined by initial values (for classical physics)

Ex. The wave equation

$$\frac{\partial^2}{\partial t^2}\phi - \frac{1}{c^2}\nabla^2\phi = 0$$

Ex. Newton's Second Law

$$\sum \vec{F} = m\vec{a}$$

Traditional Numerical Integration

- Future space only depends on previous space
- Space can be parallelized because we know its previous value
- Time has to be sequential because we don't

The discretized 1D wave equation

$$\frac{\phi_{x_j}^{t_{i+1}} - 2\phi_{x_j}^{t_i} + \phi_{x_j}^{t_{i-1}}}{\Delta t^2} = \frac{1}{c^2}\frac{\phi_{x_{j+1}}^{t_i} - 2\phi_{x_j}^{t_i} + \phi_{x_{j-1}}^{t_i}}{\Delta x^2}$$

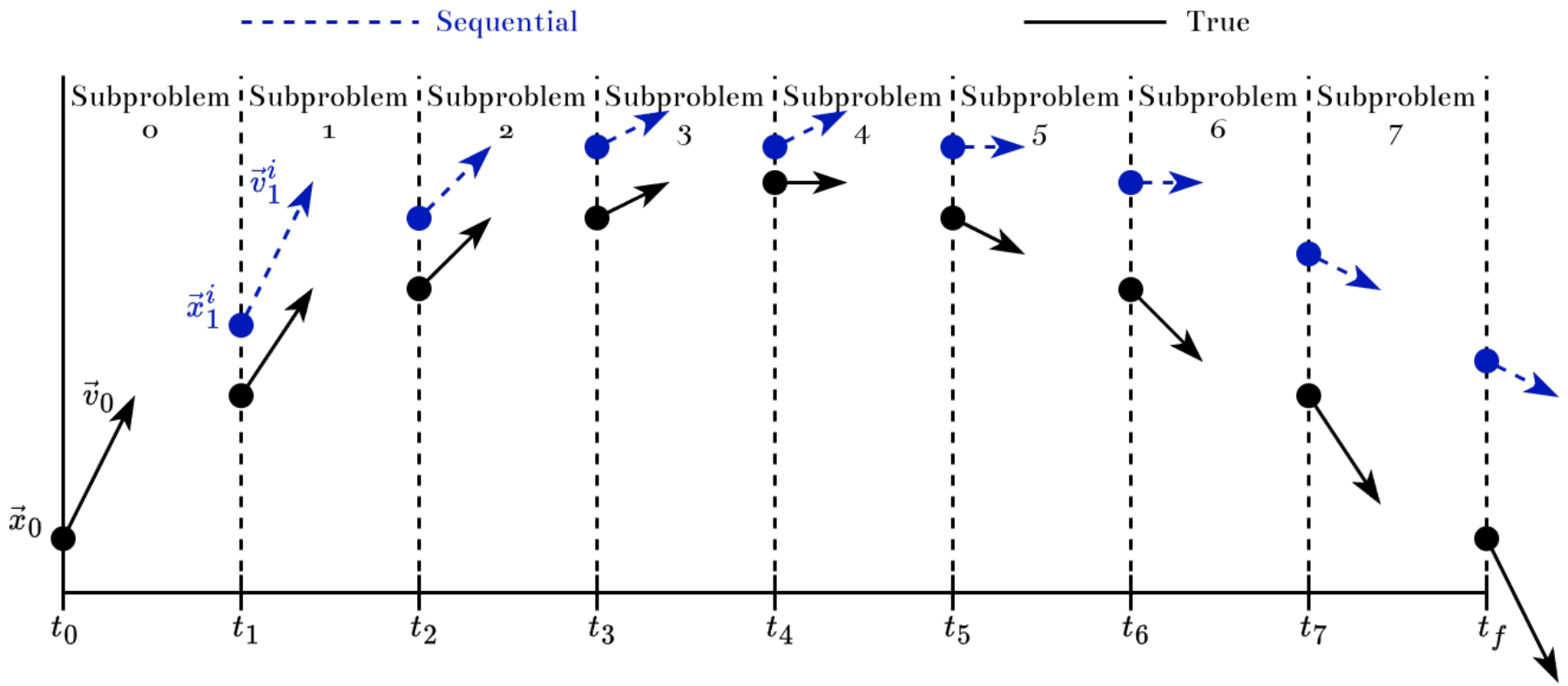
$$\implies \phi_j^{i+1} = 2\phi_j^i - \phi_j^{i-1} + \left(\frac{\Delta t}{c\Delta x}\right)^2 \left(\phi_{j+1}^i - 2\phi_j^i + \phi_{j-1}^i\right)$$

The Parareal Algorithm

- What about approximating the future?
- The Parareal Algorithm
  1. Make inaccurate prediction sequentially
  2. Make accurate predictions in parallel based on those values
  3. Correct inaccurate prediction with accurate ones
  4. Loop until converged

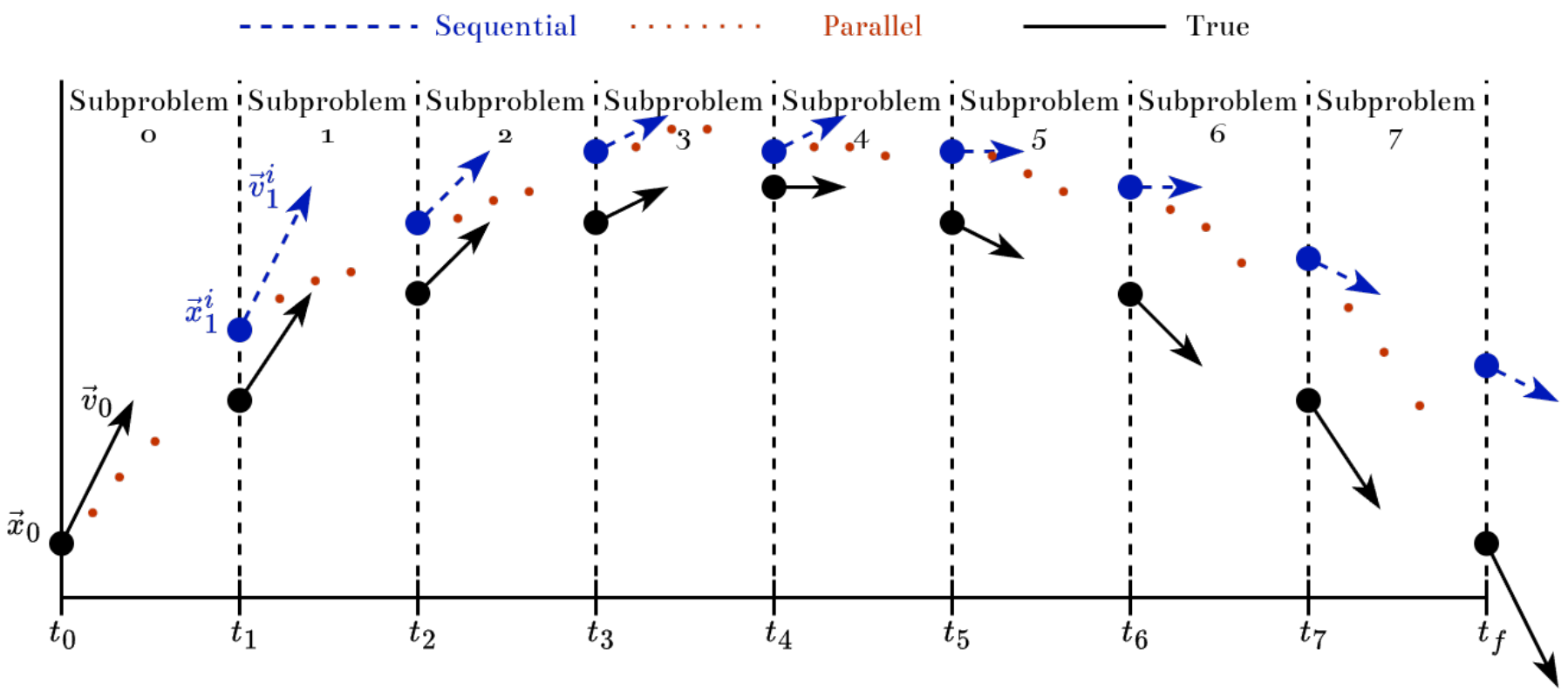
The Parareal Algorithm - Prepare the Subproblems

- Use cheap, inaccurate “coarse” solver to approximate solution



The Parareal Algorithm - Solve the Subproblems

- Use expensive, accurate “fine” solver on each coarse value



The Parareal Algorithm - Correct the Coarse Solution

- Use cheap, inaccurate "coarse" solver but correct each propagation
- Correction term = difference between previous iteration's fine and coarse values
- "Fine deviation decreases each iteration"

Literature

$$u_t^i := \underbrace{\mathcal{G}(u_{t-1}^i)}_{\text{predictor}} + \underbrace{\mathcal{F}(u_{t-1}^{i-1}) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{corrector}}$$

$$u_t^i := \underbrace{\mathcal{F}(u_{t-1}^{i-1})}_{\text{fine solution}} + \underbrace{\mathcal{G}(u_{t-1}^i) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{deviation}}$$

Alternatively

The Parareal Algorithm - Converge the Coarse Solution

- Keep iterating until coarse solution doesn't change much
- Max iterations = coarse discretization

Convergence condition

$$\max_{1 \leq t \leq N-1} |u_t^i - u_t^{i-1}| < \varepsilon$$

Content

Review of High-Performance Computing

Parareal on the GPU

Content

Numerical Analysis

Latency & Data Transfers

Content

Thanks for Listening.